

Tutorial 3

MATH10017

1 Local Variables

Open `stack.c`, you should see the following code.

```
#include <stdio.h>
int addtwo(int a) {
    a = a + 2;
    return a;
}

int multiplytwo(int a){
    a = a * 2;
    return a;
}

int main(void) {
    int a = 0;
    printf("a: %d\n", a);

    addtwo(a);
    printf("a: %d\n", a);

    a = addtwo(a);
    printf("a: %d\n", a);

    a = multiplytwo(addtwo(a));
    printf("a: %d\n", a);

    a = multiplytwo(addtwo(a) / addtwo(a));
```

```

    printf("a: %d\n", a);

    return 0;
}

```

Without running the code, what is the print out? Run the code to verify whether you got it right.

1.1 Fibonacci Sequence (revisited)

Open `fibonacci.c`, which computes the Fibonacci sequence using a loop. Modify the code to compute the sequence using a recursion rather than a loop.

Hint: you will need to define a function

```
void print_fibonacci(int previous, int current, int index, int n)
```

which prints the ‘current’ Fibonacci numbers, and the calls itself with updated arguments.

1.2 Chain Rule (Challenging)

A neural network is essentially a composite function. For simplicity, let us define a neural network with depth n as a composite function g ,

$$g(x) = f_1(f_2(\dots f_n(x))),$$

where $f_1 = f_2 = \dots = f_n = \sin$. For example, a two-layer neural network would be $\sin(\sin(x))$.

Open `chain.c`, both f and f' have been defined for you.

- Write a function `double g(double x, int n)`. It evaluates a neural network g with depth n at input x .
- Write a function `double dg(double x, int n)`. It evaluates the derivative of a neural network g with depth n at input x .

You can use loops. However, recursion would make things much simpler. Note: If you compile from a Linux terminal, link the mathematics library by placing `-lm` at the end:

```
gcc chain.c -o chain -lm
```